AutoDeploy with Docker & Jenkins

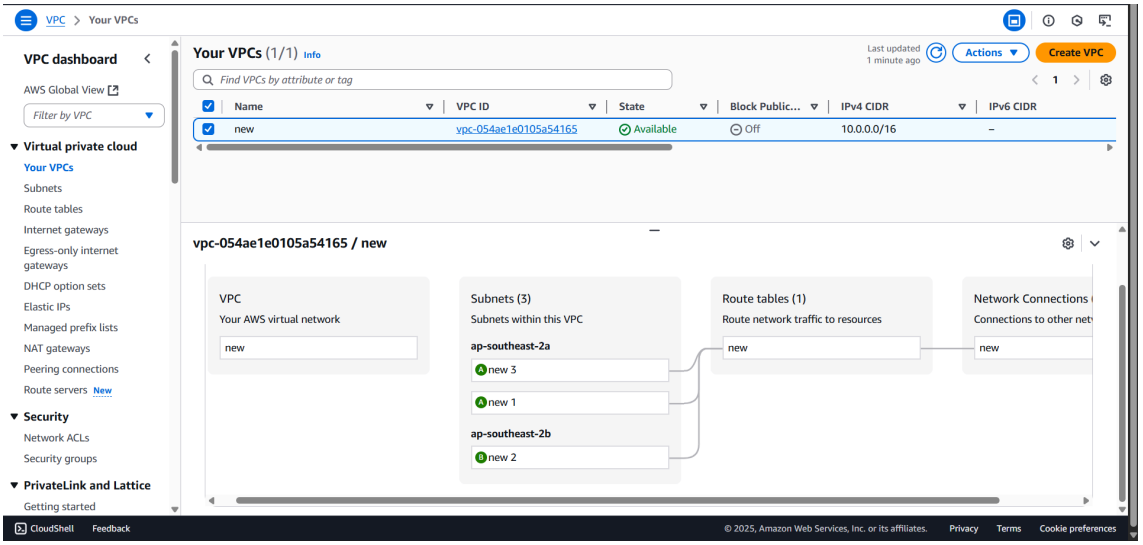
Project Overview

This project demonstrates a complete CI/CD pipeline using **AWS EC2**, **Docker**, **Jenkins**, and **GitHub**. The goal is to automate application deployment so that every time code is updated in GitHub, it automatically builds, tests, and deploys on a server.

AWS Setup (VPC, EC2):

Create a VPC (Virtual Private Cloud)

- Steps:
    1. Go to **VPC Dashboard** → **Create VPC**.
    2. Set **IPv4 CIDR block** (e.g., 10.0.0.0/16).
    3. Create **subnets** (public for EC2, private if needed).
    4. Create an **Internet Gateway** and attach it to your VPC.
    5. Update **route tables** to allow internet access for public subnet



VPC (Virtual Private Cloud)

- **What it is:** A private network in AWS.
- **Why we use it:** Keeps our EC2 instance secure and controls access to it.

Launch an EC2 Instance

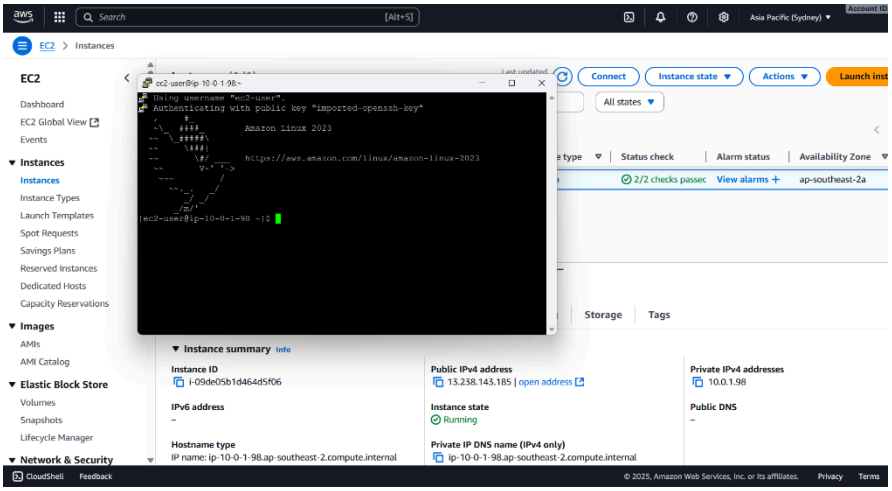
- Go to **EC2 Dashboard** → **Launch Instance**.
- Choose Ubuntu

- Attach the **security group** created
- Generate and download **key pair** (.pem) for SSH.

AWS EC2

- **What it is:** A virtual server in the cloud where you can run applications.
- **Why we use it:** Hosts our app and Docker containers so they can be accessed online.

Then Connect to EC2 via Putty:

- 

Install Docker on EC2

Docker helps run applications in container

sudo apt update

Install Docker

sudo apt install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker $USER

Verify installation:
docker –version

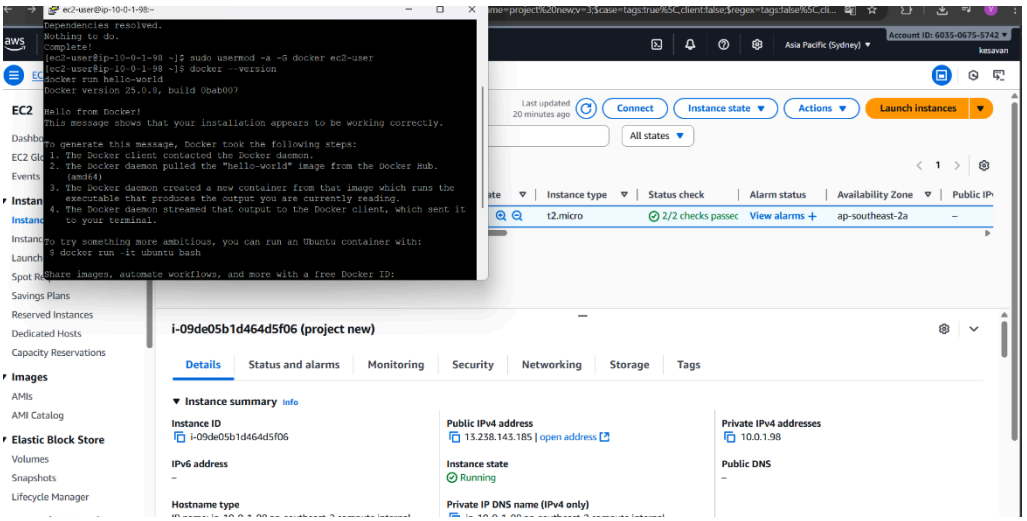
docker run hello-world

Docker

- **What it is:** A platform to create, run, and manage applications in **containers**.
- **Why we use it:** Makes sure the app runs the same way on any machine, without installation issues.

Test Docker:

docker run hello-world



Workspace on Local Repo (Laptop)

Create a New Local Repo on Desktop → My-apps (Folder) → Inside that folder, Create a Web Application file (HTML File).

Initialize and push code:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/yourusername/myapp.git
git push origin master
```

Install Git & Pull the Code on Ubuntu Instance

sudo apt install git -y

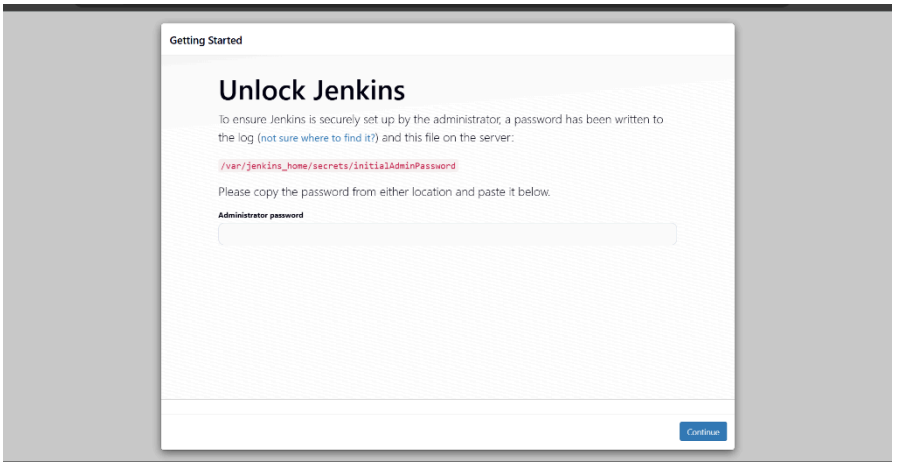
git clone https://github.com/yourusername/myapp.git

- Unlock using /var/lib/jenkins/secrets/initialAdminPassword.
- Install recommended plugins.

Install Jenkins

```
sudo apt update
sudo apt install fontconfig openjdk-21-jre
java -version
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins -y
```

Open Jenkins in browser
http://13.238.143.185:8080



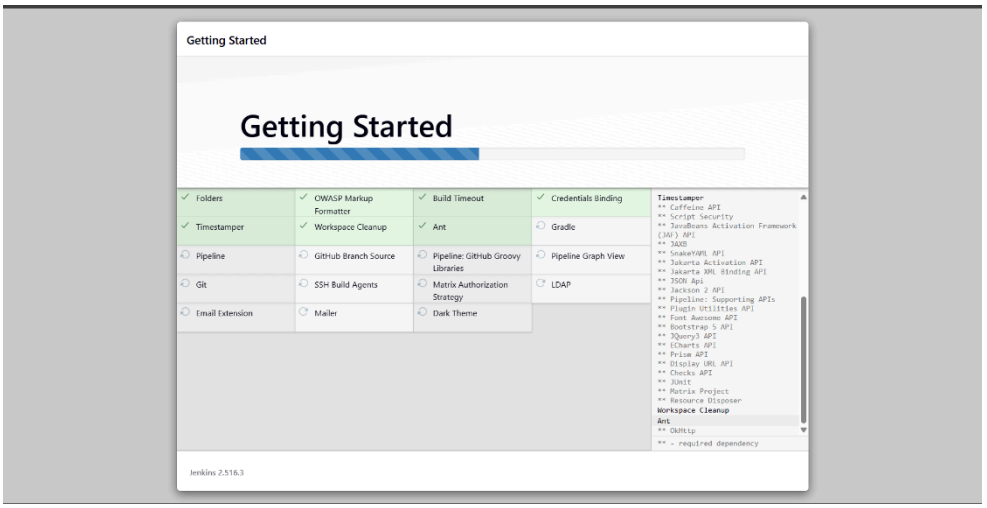
Get the initial admin password

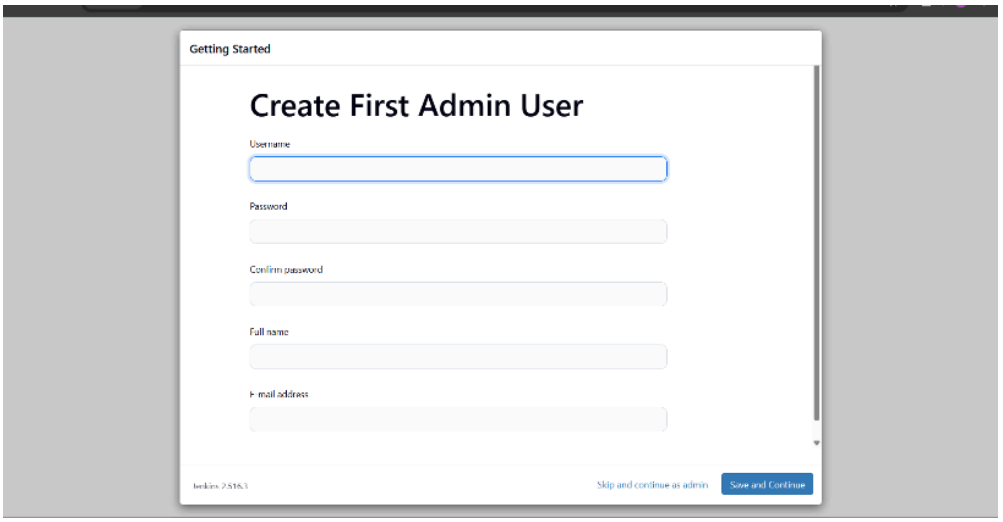
```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

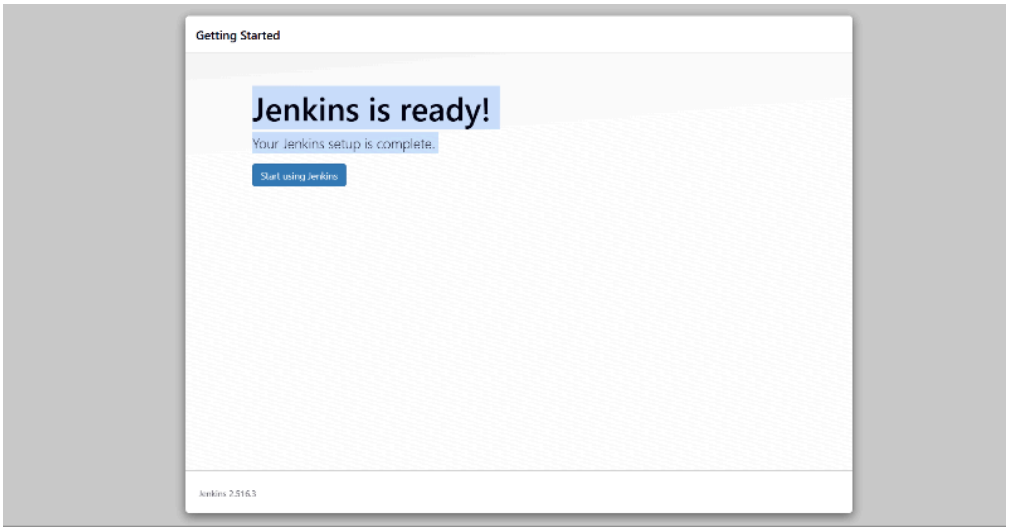
3. Jenkins

- **What it is:** An automation tool for **CI/CD** (Continuous Integration and Continuous Deployment).
- **Why we use it:** Automatically builds, tests, and deploys our app whenever we push new code to GitHub.

Then go to main page







Integrate Jenkins with GitHub:
Install **GitHub plugin** in Jenkins.

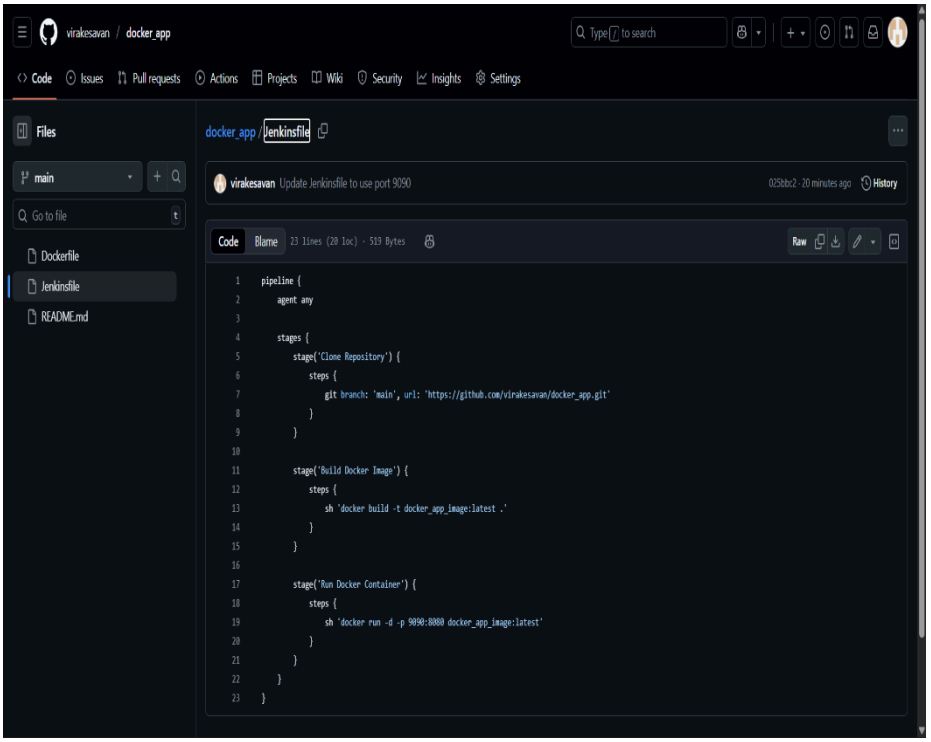
Add **GitHub credentials** in Jenkins (Personal Access Token).

Create a **New Pipeline**:

- Choose **Jenkins Pipeline script from SCM** → **Git** → **repository URL**.
- Add Jenkinsfile (in repo) to define build and deployment steps.

Git & GitHub

- **What they are:** Git is a version control system; GitHub is a cloud service to store your Git repositories.
- **Why we use them:** To manage code changes and allow Jenkins to **pull the latest code** automatically for deployment.



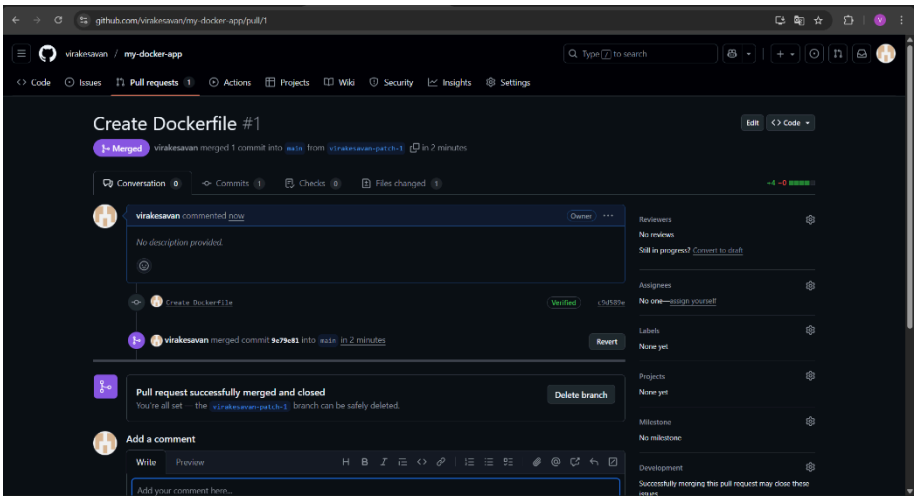
Docker Deployment

Docker builds and runs your app automatically.

Every push to GitHub triggers Jenkins:

1. Pulls latest code.
2. Builds Docker image.

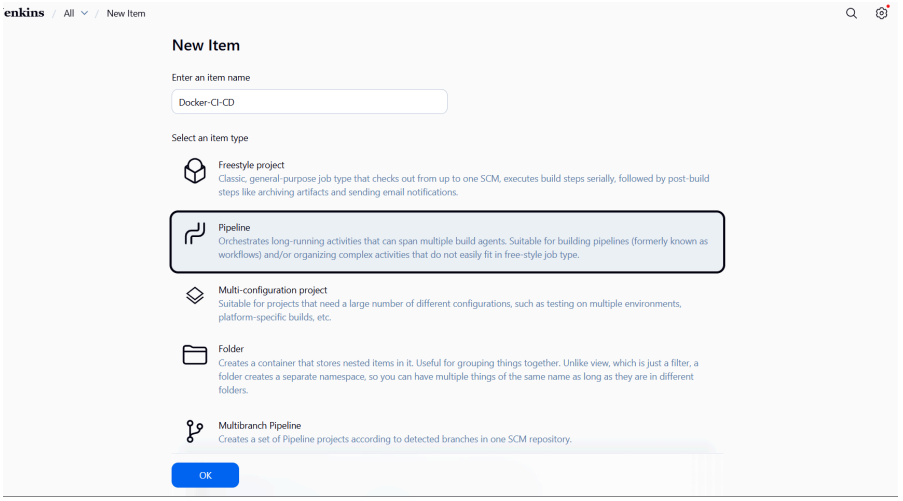
3. Runs the container.

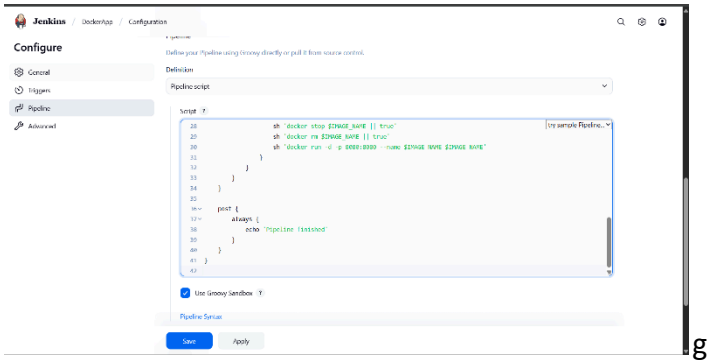


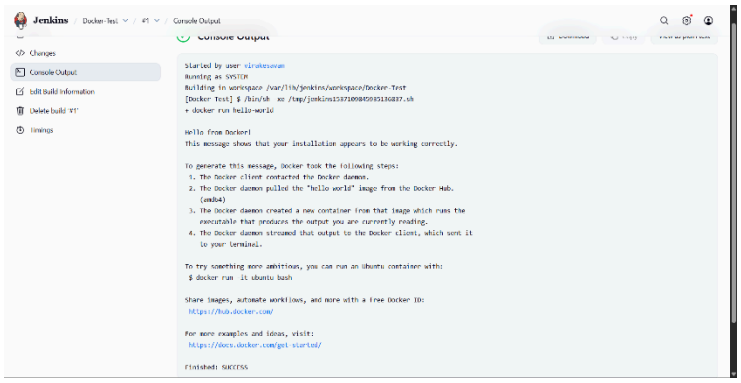
Test the Pipeline

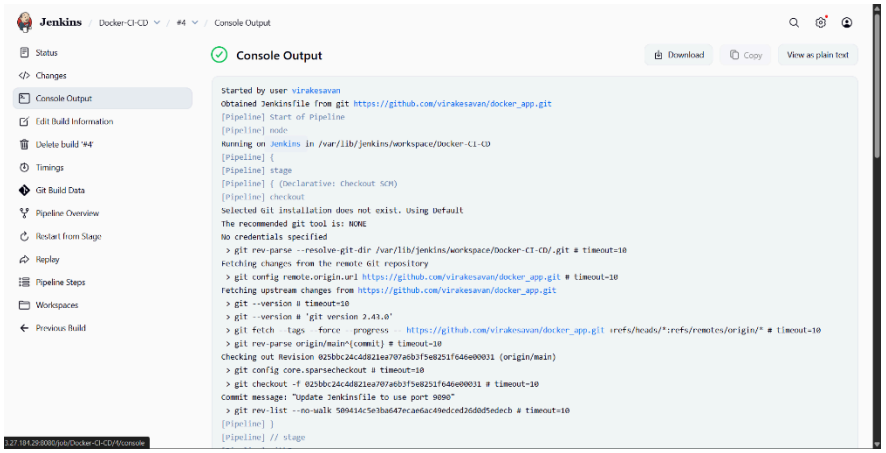
- Push a change to your GitHub repository.
- Jenkins automatically triggers the pipeline.
- Watch the console output — it should:
- Clone the repo
- Build Docker image
- Run container
- Open http:// **13.238.143.185**:5000 to see the app running.

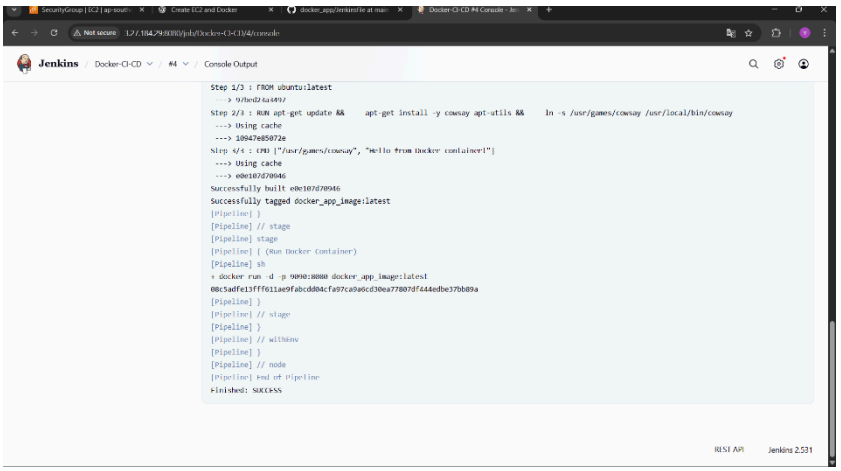
I give the screenshots that step by step process of the config




Jenkins / DockerApp / Configuration
Configure
General
Triggers
Pipeline
Advanced
Definition
Pipeline script
Script
sh 'docker stop $IMAGE_NAME || true'
sh 'docker rm $IMAGE_NAME || true'
sh 'docker run -d -p 8080:8080 --name $IMAGE_NAME $IMAGE_NAME'
post {
always {
echo 'Pipeline finished'
Use Groovy Sandbox
Pipeline Syntax
Save
Apply
g


Jenkins / Docker-Test / #1 / Console Output
Changes
Console Output
Edit Build Information
Delete build '#1'
Timings
Started by user virakesavan
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Docker-Test
[Docker Test] $ /bin/sh -xe /tmp/jenkins15371008450851368377.sh
+ docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the
executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
to your terminal.
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/
For more examples and ideas, visit:
https://docs.docker.com/get-started/
Finished: SUCCESS


Jenkins / Docker-CI-CD / #4 / Console Output
Status
Changes
Console Output
Edit Build Information
Delete build '#4'
Timings
Git Build Data
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
Console Output
Download
Copy
View as plain text
Started by user virakesavan
Obtained Jenkinsfile from git https://github.com/virakesavan/docker_app.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Docker-CI-CD
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Docker-CI-CD/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/virakesavan/docker_app.git # timeout=10
Fetching upstream changes from https://github.com/virakesavan/docker_app.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/virakesavan/docker_app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse origin/main^{commit} # timeout=10
Checking out Revision 025bbc24c4d821ea707a6b3f5e0251f646e00031 (origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 025bbc24c4d821ea707a6b3f5e0251f646e00031 # timeout=10
Commit message: "Update Jenkinsfile to use port 9090"
> git rev-list --no-walk 509414c5e3ba647ecae6ac49edced26d0d5edecb # timeout=10
[Pipeline] }
[Pipeline] // stage



Final Output

EC2 instance launched
Docker installed and running
Jenkins installed and configured
GitHub integrated
CI/CD pipeline working successfully
Application automatically deployed inside Docker container

Conclusion

In this project, I successfully implemented a CI/CD pipeline using Jenkins, Docker, and GitHub on an AWS EC2 instance.

Whenever new code is pushed to GitHub, Jenkins automatically builds a Docker image and deploys the application, minimizing manual intervention and ensuring seamless continuous integration and delivery.